

Software Development Life Cycle (SDLC)

Expanded classroom notes with detailed explanations, original diagrams, a continuous real-life example, and group class work

SDLC Life Cycle

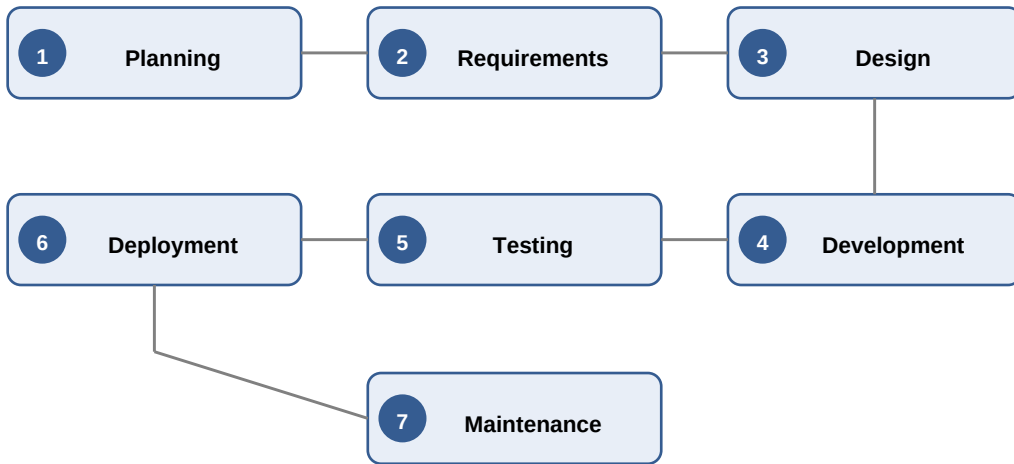


Figure 1. High-level SDLC flow used in this classroom guide.

1. What is SDLC?

Software Development Life Cycle (SDLC) is a systematic process used to plan, design, develop, test, deploy, and maintain software. It gives a software team a structured way to convert a business problem into a working product and then keep that product useful after release.

SDLC is not simply a programming checklist. It includes business understanding, requirements engineering, architecture and design, implementation, quality assurance, release management, operations, maintenance, and change management. Different development models organize these activities differently.

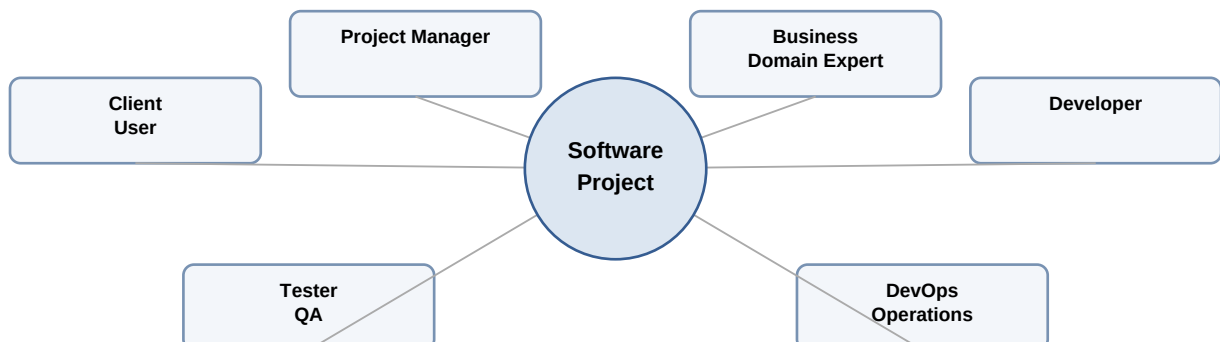
Simple idea: Understand the problem, decide what the system must do, design a solution, build it, verify it, release it, and improve it.

2. Running Real-Life Example: FoodFast

We will use a hypothetical online food-delivery application called **FoodFast**. Customers can create accounts, browse restaurants, view menus, add food to a cart, place orders, pay online, receive notifications, and track delivery.

The same example is used throughout the guide so students can see how one phase produces information needed by another phase.

Typical SDLC Project Team



3. SDLC Phase-by-Phase Explanation

1. Planning / Feasibility Study

Purpose: Determine whether the proposed project is worth pursuing and establish its initial scope, objectives, resources, risks, cost, and schedule.

What happens: The team identifies the business problem, defines objectives, estimates effort and budget, identifies stakeholders, studies technical constraints, and considers technical, economic, operational, and schedule feasibility.

FoodFast example: The FoodFast team studies customer demand, expected users, restaurant participation, payment-gateway needs, cloud infrastructure, team size, estimated cost, and target launch date. If benefits justify cost and the solution is practical, the project proceeds.

Typical deliverables: Project proposal, feasibility report, initial scope, high-level schedule, cost estimate, risk list, and project plan.

Key question: Should we build this system, and is the proposed solution feasible?

2. Requirement Analysis

Purpose: Understand exactly what customers, business owners, administrators, and other stakeholders expect from the software.

What happens: Requirements are elicited, analyzed, prioritized, documented, and validated. Functional requirements describe behavior; non-functional requirements describe qualities such as security, performance, availability, usability, and scalability.

FoodFast example: FoodFast functional requirements include registration, login, restaurant search, menu browsing, cart management, order placement, payment, cancellation, notifications, and tracking. Non-functional requirements include secure payment handling, acceptable response time, high availability, and scalability.

Typical deliverables: Software Requirements Specification (SRS), use cases or user stories, acceptance criteria, and requirement records.

Key question: What exactly must the software do, and what quality constraints must it satisfy?

3. System Design

Purpose: Convert approved requirements into a technical blueprint that developers can implement.

What happens: The team decides system architecture, modules, interfaces, database schema, API design, technology stack, security mechanisms, and important UI structures. High-level and detailed designs may be produced.

FoodFast example: For FoodFast, the team designs frontend and backend services, authentication, restaurant and menu services, cart and order modules, payment integration, delivery tracking, notifications, and database entities such as User, Restaurant, Menu, Order, Payment, and Delivery.

Typical deliverables: Architecture diagram, UML diagrams, database/ER design, API specification, UI prototypes, component design, and security design.

Key question: How should the system be structured so the requirements can be implemented reliably?

4. Development / Implementation

Purpose: Turn the approved design into executable software through programming, integration, and code-level verification.

What happens: Developers create modules, write source code, follow coding standards, use version control, perform code reviews, manage dependencies, and integrate components while remaining aligned with requirements and design.

FoodFast example: FoodFast developers implement account management, restaurant listing, menu display, cart operations, order processing, payment integration, notifications, and delivery tracking. Frontend components communicate with backend services through APIs.

Typical deliverables: Source code, builds, modules, database scripts, configuration files, unit-test results, and developer documentation.

Key question: How do we convert the design into working, maintainable software?

5. Testing

Purpose: Find defects and provide evidence that the implemented system satisfies its requirements and behaves correctly.

What happens: Testing can include unit, integration, system, regression, security, usability, performance, and user acceptance testing. Test cases are derived from requirements; defects are recorded, fixed, retested, and tracked.

FoodFast example: FoodFast testers check restaurant and menu display, cart totals, successful and failed payments, duplicate-order prevention, cancellation rules, notifications, security, and performance when many customers use the system.

Typical deliverables: Test plan, test cases, test data, defect reports, test execution results, regression results, and release recommendation.

Key question: Does the software work correctly and meet the requirements?

6. Deployment / Release

Purpose: Move tested software into production and make it available to real users.

What happens: The team prepares production infrastructure, databases, configuration, monitoring, security settings, backups, deployment scripts, and release documentation. Deployment may be direct, phased, pilot-based, or controlled rollout.

FoodFast example: FoodFast is deployed to production cloud infrastructure. The production database and payment gateway are configured, monitoring is enabled, and the application is released. The company may launch in one city before expanding.

Typical deliverables: Production build, deployment configuration, release notes, operational documentation, monitoring setup, and rollback or backup procedures.

Key question: How will the tested software be released safely and made available to users?

7. Maintenance

Purpose: Keep the software useful, secure, reliable, and compatible after release.

What happens: Maintenance includes corrective changes for defects, adaptive changes for new operating systems or external services, perfective changes for improvements, and preventive changes that reduce future problems. Monitoring and user feedback are important inputs.

FoodFast example: FoodFast may fix a payment bug, patch a security vulnerability, update the application for a new mobile OS, improve delivery tracking, optimize database queries, and add approved features such as coupons.

Typical deliverables: Patches, updates, new releases, performance improvements, security fixes, change records, and maintenance documentation.

Key question: How do we keep improving and supporting the software throughout its operational life?

4. How the SDLC Phases Are Connected

The phases should not be treated as isolated topics. Requirements provide the basis for design; design guides development; implemented functionality becomes the subject of testing; tested software is prepared for deployment; and operational feedback and defects feed maintenance.

Phase	Main Question	Major Output
Planning	Should we build it?	Feasibility and project plan
Requirements	What should it do?	SRS and requirements
Design	How should it work?	Architecture and detailed design
Development	How do we build it?	Source code and builds
Testing	Does it work correctly?	Test evidence and defect reports
Deployment	How do users get it?	Production release
Maintenance	How do we keep/improve it?	Updates, patches, enhancements

5. SDLC Models: Why the Sequence Can Change

SDLC describes the activities involved in developing and operating software, but different process models organize them differently. Waterfall largely follows a sequential structure. Agile repeats planning, requirements, design, coding, and testing in short iterations. Incremental development delivers functionality in multiple increments, while Spiral emphasizes repeated risk analysis and development cycles.

Model	Basic idea	Useful discussion point
Waterfall	Sequential phases	Stable, well-understood requirements
Incremental	Deliver functionality in increments	Early usable functionality
Spiral	Iterative development with strong risk analysis	High-risk or complex projects
Agile	Short iterations with frequent feedback	Changing requirements and rapid feedback

6. Group Class Work: Apply SDLC to a Real Project

Objective: Work as a software engineering team. Select one project and analyze it using all seven SDLC phases.

Group size: 4 to 5 students **Suggested time:** 35 to 45 minutes

Choose ONE: College Attendance Management System; Online Library Management System; Hospital Appointment Management System; Online Examination System; College Canteen Food Ordering System; Bus Ticket Booking System.

Your group must prepare:

1. Write a short problem statement: What problem is the software solving?
2. Identify at least 6 stakeholders or user types.
3. Write at least 8 functional requirements.
4. Write at least 4 non-functional requirements.
5. Explain what the team would do in each of the seven SDLC phases.
6. Give at least one concrete real-life example for every phase.
7. List at least two deliverables produced by each phase.
8. Draw an SDLC flow diagram for your project.
9. Identify two risks and explain where they may be discovered.
10. Select the most suitable SDLC model and give two reasons.

Suggested Group Roles

Member	Suggested responsibility
1	Project coordinator and problem statement
2	Requirements and stakeholder analysis
3	System design and SDLC diagram
4	Testing, risks, and quality analysis
5	Presentation and final integration

Presentation: 5 to 7 minutes per group. Every member should speak. Include the problem, stakeholders, requirements, seven SDLC phases, examples, selected model, and justification.

7. Quick Classroom Check

1. What is the purpose of SDLC?
2. What is the difference between functional and non-functional requirements?
3. Why is an SRS important?
4. Which phase converts requirements into a technical blueprint?
5. Why is testing performed before deployment?
6. Give one example of corrective maintenance.
7. Give one example of adaptive maintenance.
8. Why might Agile repeat requirements, design, coding, and testing?
9. What could happen if a major requirement is misunderstood?
10. Which SDLC model would you choose for rapidly changing requirements, and why?